

- 参考资料
 - 1. 初始值确认（16进制）
 - 2. 修正后的实验步骤表
 - 重点步骤解释（第5步发生了什么？）
 - 最终结果验证
- 一些问题
 - 1. 为什么要重置DRA为1？
 - 2. 执行移位的时候DRA和DRB的移位顺序
 - 问题1: 判断Qn的本质
- 🧩 深入分析题
 - 问题3: 追踪数据流
 - 问题4: 换个乘数试试
 - 问题5: 寄存器的角色
- 🚀 综合应用题(挑战)
 - 问题6: 算法效率分析
 - 问题7: 边界情况
 - 问题8: 反推设计
- 🎓 终极理解题
 - 问题9: 本质理解
 - 问题10: 代码实现

参考资料

[【计算机组成原理】二进制数的乘法运算_哔哩哔哩_bilibili](#)

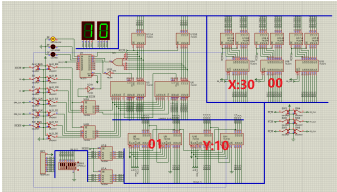
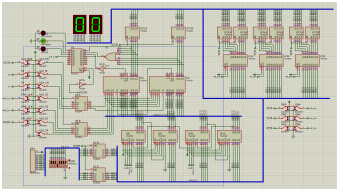
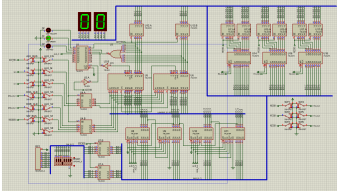
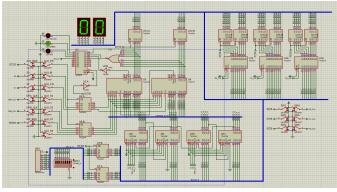
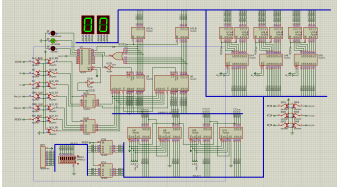
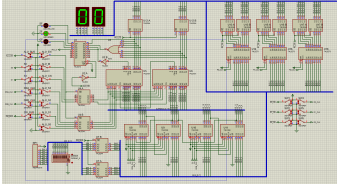
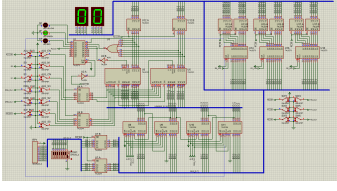
1. 初始值确认（16进制）

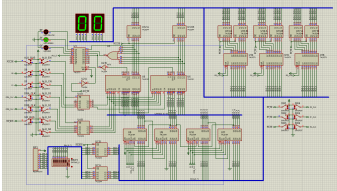
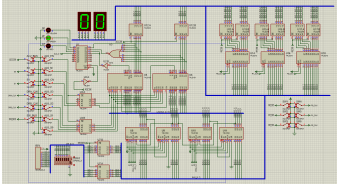
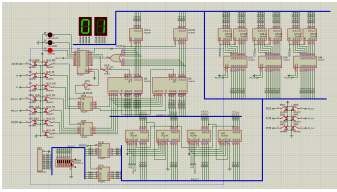
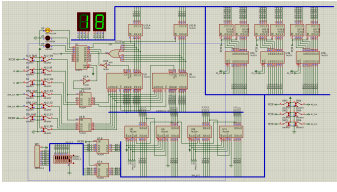
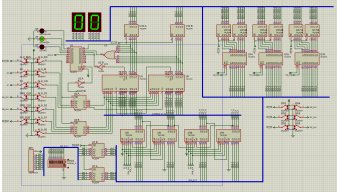
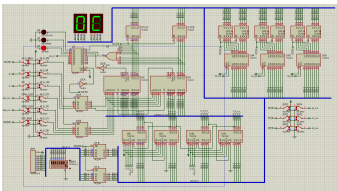
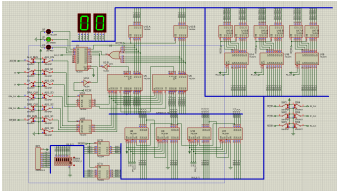
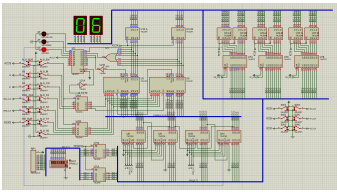
- 被乘数 X (REG_0): $30_{(16)}$
 - 二进制: 0011 0000
- 乘数 Y (DRB): $10_{(16)}$
 - 二进制: 0001 0000 (注意这里！只有第5位是1，其他全是0)
- 预期结果: $30_{(16)} \times 10_{(16)} = 300_{(16)}$
 - 这就像十进制里 $30 \times 10 = 300$ 一样，16进制里 30×10 也是 300。

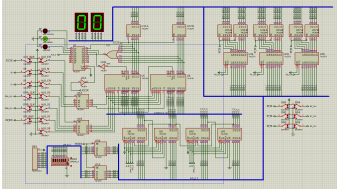
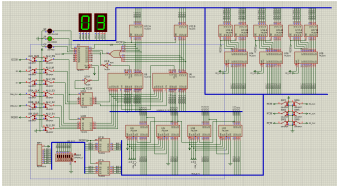
2. 修正后的实验步骤表

关键逻辑更新：

- $Y = 10$ (0001 0000) 的最低位（第1位）是0。
- 它右移4次后，那个 1 才会移动到最低位（ $Q_n=1$ ），触发加法。
- 所以会在 第5步 进行加法，其余步骤全是移位。

步骤	DRA (高位/部分积)	DRB (低位/乘数)	REG_0 (被乘数 X)	REG_1 (累加器)	REG_2 (备份 Y)	Qn (判断位)	截图
设置初值	0000 0001	10	30	00	00	-	
01 (判断)	00	10	30	00	00	0	
移位结果	00	08	30	00	00	-	
02 (判断)	00	08	30	00	00	0	
移位结果	00	04	30	00	00	-	
03 (判断)	00	04	30	00	00	0	
移位结果	00	02	30	00	00	-	

步骤	DRA (高位/部分积)	DRB (低位/乘数)	REG_0 (被乘数 X)	REG_1 (累加器)	REG_2 (备份 Y)	Qn (判断位)	截图
04 (判断)	00	02	30	00	00	0	
移位结果	00	01	30	00	00	-	
05 (判断)	30 (加法后)	01	30	30	01	1	
移位结果	18	00	30	18	01	-	
06 (判断)	18	00	30	18	01	0	
移位结果	0C	00	30	0C	01	-	
07 (判断)	0C	00	30	0C	01	0	
移位结果	06	00	30	06	01	-	

步骤	DRA (高位/部分积)	DRB (低位/乘数)	REG_0 (被乘数 X)	REG_1 (累加器)	REG_2 (备份 Y)	Qn (判断位)	截图
08 (判断)	06	00	30	06	01	0	
移位结果	03	00	30	03	01	-	
最终结果:0000 0011 0000 0000							

非常幸运,因为乘数是01,只用加一次

重点步骤解释（第5步发生了什么？）

- 判断 $Q_n=1$:
 - 在第5步开始时, DRB 已经被移位成了 01 (0000 0001)。
 - 最低位是 1, 所以 $Q_n=1$, 触发加法逻辑。
- 执行加法:
 - 部分积 (REG_1) 00 + 被乘数 (REG_0) 30 = 30。
 - 结果 30 存入 DRA 和 REG_1。
- 执行移位:
 - DRA (30) 右移:
 - 30 (hex) = 0011 0000 (bin)。
 - 右移一位变成 0001 1000 = 18 (hex)。
 - 注意: 30 的最低位是 0, 所以移出的位是 0。
 - DRB (01) 右移:
 - DRB 接收来自 DRA 移出的那个位 (也就是 0)。
 - 所以 01 右移并补入 0 $\rightarrow$ 00。

4. 后续步骤:

- 之后 DRB 已经是 00 了, 所有 Qn 都是 0, 只是一直把高位的 18 往右移:
- 18 → 0C → 06 → 03。

最终结果验证

- DRA (高8位): 03
- DRB (低8位): 00
- 拼接结果: 0300 (16进制)
- 符合预期: $30_{16} \times 10_{16} = 300_{16}$ 。

一些问题

1. 为什么要重置DRA为1?

DRA是多功能寄存器,在中间被"污染"了

原因	解释
1. DRA是多功能寄存器	既用来判断Qn,又用来存储加法结果
2. 判断需要固定模式	必须用"DRB & 1"来提取最低位
3. 每轮循环要独立	上一轮的计算结果不能影响下一轮的判断

2. 执行移位的时候DRA和DRB的移位顺序

我们需要先移动DRB再移动DRA不然的话DRB最高位可能会被DRA最低位污染导致计算错误

问题1: 判断Qn的本质

Q: 为什么用 DRB & 00000001 可以判断Qn?如果我用 DRB & 00000010 会判断什么?

► 💡 提示

判断乘数的最低位,看是否需要相加,为0不用加,为1需要加 ### 问题2: 移位顺序的重要性

Q: 实验中强调"先移DRB,再移DRA",如果顺序反过来会发生什么错误?请举例说明。

► 💡 提示

我们需要先移动DRB再移动DRA不然的话DRB最高位可能会被DRA最低位污染导致计算错误

深入分析题

问题3: 追踪数据流

Q: 在第5步加法后,DRA变成了 30_{16} 。请问:

- 这个30是怎么来的?(哪两个数相加?)
- 为什么是存入DRA而不是其他寄存器?
- 移位后变成 18_{16} ,请用二进制解释为什么 $30 \rightarrow 18$? 就是我们的被乘数30和我们的累加计数器00相加的

问题4: 换个乘数试试

Q: 如果乘数改成 11_{16} (0001 0001),预测:

- 哪几步会触发加法($Q_n=1$)?
- 最终结果是多少?
- 画出关键步骤的DRA和DRB变化

► 💡 提示

两次乘法 $\$330_{16}$

问题5: 寄存器的角色

Q: 实验中有5个寄存器:REG_0, REG_1, REG_2, DRA, DRB。请说明:

- 哪个始终不变?
- 哪个每轮都要重置?
- 哪个存储最终结果?

- REG_2的作用是什么?(为什么需要它?) REG_0 判断的时候DRA每轮都需要重置DRA,DRB存储最终结果 REG2用于临时存储乘数

综合应用题(挑战)

问题6: 算法效率分析

Q: 为什么说这个算法"幸运"?如果乘数是 FF_{16} (1111 1111):

- 需要执行几次加法?
- 哪个步骤会变慢?
- 你能想到优化方法吗? 8次, 每次都会计算加法,判断之后的加法会变慢? 各位相乘再相加,但是电路可能会非常复杂

问题7: 边界情况

Q: 考虑特殊情况:

- 情况A: 被乘数=00, 乘数=10,结果会对吗?
- 情况B: 被乘数=FF, 乘数=FF,会溢出吗?如何检测?
- 情况C: 乘数=00,算法会提前结束吗? A:会对 B:不会溢出,乘出的结果最多4为16进制数 C:不会

问题8: 反推设计

Q: 假设你要设计一个串行除法器,能否参考乘法器的思路?

- 判断位应该判断什么?
- 需要执行什么运算?(加法?减法?)
- 移位方向是左还是右?

判断是否减法溢出? 减法 左移?

终极理解题

问题9: 本质理解

Q: 用一句话解释:为什么乘法可以转化为"判断-加法-移位"的循环?

这个算法本质上在模拟什么数学原理?

► 💡 提示

乘法本质上就是多个步骤的加法

问题10: 代码实现

Q: 如果让你用Python实现这个算法,写出核心循环的伪代码:

```
def serial_multiply(X, Y):  
    # 初始化  
    REG_0 = X # 被乘数  
    REG_1 = 0 # 累加器  
    DRB = Y   # 乘数  
  
    for i in range(8): # 8位乘法  
  
    return result
```